

Diseña e implementa una aplicación web

Proyecto

Cambios v2:

- Se ha añadido al enunciado la Práctica 1.

Cambios v3:

- Se han movido las instrucciones del vídeo de la Práctica 1 a la rúbrica de dicha práctica.

Cambios v4:

- Se ha añadido al enunciado la Práctica 2.

Cambios v5

- Se ha añadido una clarificación en el script de creación de la imagen Docker y se han añadido instrucciones para implementar los modos de despliegue solicitados.

Cambios v6:

- Se ha indicado que cada uno de los alumnos debe subir el Docker Compose a su cuenta de Docker Hub.

Cambios v7:

- Se ha añadido el enunciado de la Práctica 3.

NOTA: El contenido de este enunciado puede actualizarse y completarse a medida que avanza el curso. Cualquier actualización será notificada en los foros de la asignatura en el Aula Virtual.

Objetivo

Que el alumno implemente una aplicación web de la forma más parecida a como lo haría a nivel profesional. Algunas de las características de la práctica que simulan un entorno real son las siguientes:

- La práctica se desarrollará en un equipo formado por 4 o 5 alumnos. De esa forma el equipo será similar al que se puede encontrar en un desarrollo web profesional.
- Se utilizarán herramientas profesionales, tanto para el desarrollo en sí como para compartir

el código entre los miembros del equipo (IntelliJ, Visual Studio Code, GitHub, Docker...).

- La temática de la web podrá ser elegida libremente por los alumnos de cada equipo. De esta forma, los alumnos estarán más motivados y podrán incluir en la aplicación aquellas funcionalidades que deseen (dentro de unos límites que se definirán más adelante en este enunciado).

Temática de la aplicación web

La temática de la aplicación web que hay que diseñar e implementar será elegida libremente por los miembros de cada equipo. Cada equipo puede decidir qué funcionalidades ofrece al usuario, el aspecto gráfico, el esquema de navegación, etc.

A modo de ejemplo, se presentan algunos tipos de aplicaciones web que se podrían implementar, pero el equipo podrá elegir cualquier temática (aunque no esté aquí listada):

- Web de compra/venta de objetos usados (listado, proceso de compra, registro de compras, comunicación entre comprador y vendedor).
- Web de selección/contratación de cuidadores de niños por horas (listado, valoraciones/reputación, filtrado por disponibilidad, características...).
- Web de un gimnasio con seguimiento de entrenamiento (clases colectivas, horarios, registro, información pública...).
- Web de una liga de fútbol / torneo de pádel (equipos, jugadores, partidos, clasificación, calendario...).
- Web de gestión docente de una universidad (horarios, fechas de exámenes, asignación de profesores a asignaturas, fichas de asignaturas, alumnos...).
- Web de un ayuntamiento (noticias, votaciones, registrarse en actividades, calendario de actividades...).
- Web para diseñar viajes turísticos (lugares de interés, planificación, mapas, horarios...).
- Web de una academia de formación (cursos ofertados, activos, reserva, matrícula...).
- Web de venta online (productos, carrito de la compra, stock, pedido, análisis de ventas...).
- Web de reserva de aulas de informática en la universidad (registro de software por aula, calendario, conflictos, gestión manual...).
- Web para formación (tipo Moodle) (asignaturas/cursos por alumno, material trabajo, entrega de trabajos, foro...).
- Web para seguimiento y evaluación de una práctica por fases: (equipos, notas del equipo y de cada integrante, lista de items de corrección, análisis de datos, fechas...).

Es posible copiar alguna web que esté disponible en Internet, por ejemplo, una web de noticias como menéame, un gestor de blogs, la página web de consulta de horarios de la universidad, etc.

En caso de que copie una web existente, se podrán copiar también su diseño gráfico, logotipos, iconos, etc.

Metodología de desarrollo

El proyecto deberá realizarse siguiendo la siguiente metodología de desarrollo.

Desarrollo colaborativo

La aplicación web se desarrollará usando un repositorio de la plataforma GitHub¹. El código fuente de la aplicación tendrá la licencia Apache 2. El repositorio será creado por los alumnos bajo las indicaciones del profesor.

Durante el desarrollo de la aplicación se irá subiendo el código al repositorio a medida que se vaya desarrollando. Es importante que se sigan las buenas prácticas y los commits no sean muy grandes. Es decir, no se considerará adecuado que una aplicación se implemente con un commit que añada decenas de ficheros nuevos. El repositorio no sólo se utilizará para entregar la aplicación, sino que debe usarse para desarrollar la aplicación. Idealmente deberá realizarse un commit del menor tamaño posible que deje la aplicación en un estado estable (que compile). Como mucho, habrá un commit por funcionalidad.

Los mensajes de commit también tienen que ser adecuados y describir correctamente el objetivo del commit. Se recomienda que cada mensaje tenga en su primera línea un resumen del commit de no más de 50 caracteres y opcionalmente una descripción más extensa a partir de la tercera línea (dejando una línea en blanco entre el resumen y la descripción). Existen muchas páginas web con recomendaciones sobre cómo hacer buenos commits en Git²³.

Para gestionar el repositorio de GitHub se puede usar cualquier cliente de Git.

Los miembros del equipo de desarrollo pueden coordinarse entre sí como consideren conveniente, aunque es recomendable que utilicen un tablero Trello⁴ o el servicio de GitHub Projects⁵ para gestionar el estado de las tareas y su responsable.

Desarrollo

¹ <https://github.com>

² <https://github.com/trein/dev-best-practices/wiki/Git-Commit-Best-Practices>

³ <https://github.com/erlang/otp/wiki/writing-good-commit-messages>

⁴ <https://trello.com>

⁵ <https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>

El proyecto será desarrollado en diferentes prácticas (partes de la misma aplicación). De esa forma, los alumnos realizarán un trabajo continuo a lo largo del curso y el profesor podrá hacer un mejor seguimiento de este.

Las prácticas en las que se divide el desarrollo de la aplicación web son:

- Preparación 1: Definición de las funcionalidades de la web.
- Preparación 2: Maquetación de páginas web con HTML y CSS.
- Práctica 1: Web con HTML generado en servidor y AJAX.
- Práctica 2: API REST a la aplicación web, despliegue con docker y despliegue remoto.
- Práctica 3: Web con arquitectura SPA.

El contenido de cada una de estas prácticas se describe en detalle más adelante en el enunciado del proyecto.

El código de cada una de las prácticas deberá estar en el repositorio de código en las siguientes fechas antes de que comience la clase:

- Preparación 1: 02 de febrero 2026
- Preparación 2: 09 de febrero de 2026
- Práctica 1: 02 de marzo de 2026
- Práctica 2: 24 de marzo de 2026
- Práctica 3: 28 de abril de 2026

Cuando el código del repositorio esté listo para la entrega de una práctica, será necesario hacer un tag en el repositorio con el nombre de la práctica que se vaya a entregar. Por ejemplo, al entregar la Práctica 1 se deberá crear un tag llamado "parctica1", que será el que será evaluado. Los tags se pueden hacer desde la interfaz web de GitHub al crear una "release". También se deberá subir dicho tag en formato zip al aula virtual.

IMPORTANTE: No se evaluará el código de la rama main. Si no se ha creado el tag se considera que la práctica no se ha entregado.

Documentación

La documentación del proyecto se incluirá en el fichero README.md en la raíz del repositorio de código. Este fichero se editará usando adecuadamente el formato Markdown (títulos, negritas/cursivas, tablas, texto en formato código, etc.). Se debe verificar que el documento README.md se visualiza correctamente desde la web de GitHub.

En la descripción de cada una de las partes se indica de forma detallada el contenido de la documentación solicitada.

Evaluación

Cada una de las prácticas será evaluada mediante una vídeo-demostración y una defensa presencial en horario de clase. Dicha defensa consistirá en una prueba individual en la que el alumno cargará la práctica en el ordenador del aula y realizará una o varias tareas sobre la misma, tales como explicaciones, modificaciones o adiciones. Al tratarse de una actividad de evaluación, todos los alumnos del equipo deberán estar presentes en la defensa, incluso aquellos que dispongan de dispensa académica.

Preparación 1 (no evaluable)

La preparación 1 no lleva asociada calificación porque consiste únicamente en definir la funcionalidad de la web y la creación del equipo.

En la presentación el profesor verificará que la funcionalidad de la web definida por los alumnos cumple con los requisitos exigidos en el enunciado. Si se identifican carencias serán indicadas a los alumnos para que las solventen antes de comenzar con la siguiente práctica.

Preparación 2 (no evaluable)

La preparación 2 no lleva asociada calificación porque se asume que los alumnos ya han adquirido los conocimientos necesarios para desarrollar esta práctica en la asignatura “Fundamentos de la web” de 2º curso.

En la presentación el profesor verificará que el diseño de las pantallas y la navegación entre ellas cumple con los requisitos exigidos en el enunciado. Si se identifican carencias serán indicadas a los alumnos para que las solventen antes de comenzar con la siguiente práctica.

Evaluación de las prácticas 1, 2 y 3

La evaluación de las prácticas 1, 2 y 3 llevará asociada una calificación. La evaluación de cada práctica se realizará en dos partes:

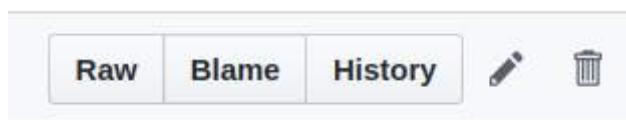
- **Vídeo-demostración:** Se realizará un vídeo de demostración, sin cortes, en el que se muestre el uso de la aplicación como lo haría un usuario real. El vídeo consistirá en un recorrido por la aplicación, presentando sus funcionalidades más relevantes para los distintos tipos de usuario: anónimo, registrado y administrador. Asimismo, el vídeo deberá mostrar cada uno de los puntos indicados en la rúbrica de la práctica correspondiente. El profesor evaluará cada funcionalidad indicada en la rúbrica con el vídeo. Si la funcionalidad no aparece o presenta errores, se aplicará la penalización correspondiente indicada en la rúbrica.
- **Defensa presencial:** Cada alumno del grupo cargará su práctica en los ordenadores del aula y realizará una o varias tareas sobre la misma, tales como explicaciones,

modificaciones o adiciones. Una vez finalizada la defensa, el código se subirá a la tarea correspondiente del aula virtual para que el profesor pueda evaluarlo y asignar la calificación.

Participación de los miembros del equipo

Todos los miembros del equipo tienen que participar activamente en el desarrollo de cada una de las prácticas. Para poder justificar su participación, cada uno de los miembros del equipo debe incluir en el README.md:

- Un párrafo describiendo de forma textual las tareas realizadas en esa práctica.
- Listado de los 5 commits más significativos durante la práctica. Cada uno de esos commits deberá ser un link que lleve a este commit en la interfaz web de GitHub.
- Listado de los 5 ficheros en los que más haya participado el miembro. Se podrá demostrar su participación al usar la opción “blame” de GitHub al visualizar ese fichero.



Si el profesor detecta una baja participación de un alumno concreto, podrá reducirse la nota de ese alumno y podría llegar a **suspender** dicha práctica.

Aunque el trabajo es colaborativo y es importante el reparto de tareas, todos los miembros del equipo deberán conocer el código completo de la aplicación (aunque algunas partes hayan sido desarrolladas por otro compañero).

Calidad del código fuente

Se exigirá una mínima calidad del código fuente. Al menos deberán tenerse en cuenta los siguientes aspectos:

- El código deberá estar formateado correctamente y usar las mismas reglas de estilo en todos los ficheros (se recomienda configurar el entorno de desarrollo para formatear al guardar y así evitar problemas de este tipo).
- El código y los comentarios de este deberán estar escritos completamente en inglés. Sólo podrán existir términos en castellano en el código cuando vayan a mostrarse en el interfaz de usuario. La existencia de texto en castellano en el código llevará aparejada una penalización en la calificación.
- Las variables, parámetros, atributos, clases e interfaces deberán tener nombres adecuados que describan su objetivo.

- Se evitará el código duplicado. Cuando dos fragmentos de código sean similares, se utilizarán las técnicas adecuadas para fomentar la reutilización (herencia, composición, subprogramación...)
- Los métodos no serán muy largos y no tendrán mucha complejidad ciclomática.
- El código deberá ser razonablemente eficiente. Por ejemplo, no se considerará válido hacer una consulta a la base de datos para obtener una lista de elementos y luego filtrar esa lista en memoria usando código Java.
- Si se quiere mostrar texto de depuración en la consola del proceso Java, se usará la librería de logs en vez de usar `System.out.println()`.

Si se incumple alguna de estas buenas prácticas la calificación podrá verse penalizada hasta en **2 puntos** dependiendo de su grado.

Calificación

Al evaluar cada una de las prácticas se tendrán en cuenta los siguientes aspectos:

- Las prácticas 1, 2 y 3 tendrán una calificación entre 0 y 10.
- Si la práctica no cumple con los mínimos exigidos (falta de funcionalidad, incompleta) o no funciona correctamente (tiene errores) se considerará suspensa. El equipo que suspenda una entrega podrá realizar las entregas de las sucesivas prácticas (subsando los errores de la entrega suspensa) y recuperar la práctica suspensa en la convocatoria ordinaria y/o extraordinaria.
- Si el profesor detecta carencias menores durante la evaluación del vídeo podrá poner una nota condicionada a que esos aspectos sean subsanados en un plazo de 1 semana. Si no son subsanados, la nota final para esa práctica sería inferior a la nota indicada.
- La nota de cada alumno del equipo es independiente. Un alumno puede obtener más o menos nota que el resto de sus compañeros. La nota individual de cada alumno se determina en base a:
 - o La nota obtenida en la defensa presencial (50 % del total).
 - o La contribución de cada miembro en esa práctica (50 % del total). Se evaluará en función de los commits reportados (siguiendo la sección "Participación de los miembros del equipo" por cada uno de los participantes en el repositorio GitHub. Se puede llegar al extremo de suspender la práctica a un miembro del equipo si no ha colaborado de forma mínima en la práctica.

Convocatoria extraordinaria

Aquellas prácticas aprobadas en la convocatoria ordinaria se guardan para la convocatoria

extraordinaria. Es decir, en la convocatoria extraordinaria los alumnos sólo tendrán que realizar aquellas prácticas suspensas o no entregadas.

La nota final de la práctica se calculará con los mismos porcentajes que en la convocatoria ordinaria.

En la convocatoria extraordinaria si no se cumplen con las funcionalidades o la aplicación tiene fallos, la práctica (y por tanto la asignatura) se considerará suspensa.

Modificación de miembros del equipo

Aunque se debería evitar en la medida de lo posible, los equipos pueden tener modificaciones en sus miembros debido a las siguientes causas:

- **División del equipo en dos debido a conflictos entre los miembros:** En ese caso, el equipo se puede dividir en 2 y cada nuevo equipo podrá seguir el desarrollo de la práctica partiendo del código en el momento de la división. Hay que evitar esta situación por el incremento de trabajo que supondría para los dos nuevos equipos.
- **Alumno abandona un equipo por conflictos:** Se podrá unir a otro equipo siempre y cuando tenga menos de 4 alumnos y tenga aprobadas las mismas prácticas que el alumno que quiere incorporarse. Es decir, un alumno no se puede ir a un equipo que tenga aprobadas las prácticas 2 y 3 si él no tiene aprobada la práctica 2 en su equipo original.
- **Alumno deja la asignatura:** Se considera que el alumno ya no forma parte del equipo.

Preparación 1: Definición de las funcionalidades de la web

En la preparación 1 se deben realizar las siguientes tareas:

- Formar el equipo de desarrollo.
- Definir las funcionalidades de la aplicación web.

Esta práctica no es evaluable.

Requisitos de la aplicación web

Para que la complejidad sea similar para todos los equipos, la aplicación web se deberán diseñar de forma que cumpla con los siguientes requisitos:

- **Entidades:** Una entidad representa un concepto que la aplicación web guarda en la base de datos. Son las clases de dominio, las tablas de la base de datos. La aplicación web deberá gestionar 4 entidades. Una de ellas será la entidad Usuario (para guardar la

información de los usuarios que acceden a la web). Las otras 3 entidades dependen de la temática de la web. Es importante que todas las entidades estén relacionadas entre sí. Por ejemplo:

- o Web de gestión de torneos o ligas: Entidades usuario, equipo, torneo y partido. La relación se tiene porque un equipo juega en partidos que forman parte de un torneo.
- o Web de gestión de cursos: Entidades usuario, curso, material, mensaje. La relación se tiene porque un alumno está en un curso, que tiene materiales y mensajes en el foro.
- o Web de cuidado de niños: Entidades usuario (progenitor o cuidador), solicitud, jornada de cuidado. La relación se tiene porque una jornada de trabajo ha sido realizada a un progenitor por un cuidador. Además, el progenitor puede poner una solicitud, que se puede convertir en una jornada realizada por un cuidador.
- o Web de gestión de exámenes: usuario (profesor o alumno), examen, asignatura, realización de examen. Las entidades están relacionadas porque un profesor imparte una asignatura con alumnos. Esos alumnos tendrán que realizar el examen de la asignatura.
- **Tipos de usuarios:** La aplicación web deberá considerar tres tipos de usuarios:
 - o **Usuario anónimo:** Aquel usuario que visita la web y no introduce ningún tipo de credenciales para consultar contenido y realizar búsquedas. Salvo que esté justificado por el tipo de aplicación, este usuario sólo podrá consultar información de la web, pero no podrá crearla ni modificarla.
 - o **Usuario registrado:** Aquel usuario que tiene que usar sus credenciales para acceder a la web. Este usuario tendrá datos personalizados como su nombre, una imagen, el histórico de acciones realizadas en la web (mensajes en foros, compras, etc..). La web deberá permitir el registro de nuevos usuarios.
 - o **Usuario administrador:** Aquel usuario que tiene control total sobre la información de la web. Por ejemplo, el alta de productos, la creación de los campeonatos, registro de información o actividades, etc. La web tendrá un único usuario administrador con una contraseña especificada en un fichero de configuración (cifrada).
- **Permisos de los usuarios:** La web tiene que estar diseñada para que los usuarios registrados puedan ser dueños de ciertos datos. Por ejemplo, si se trata de una tienda, los pedidos previos que ha realizado. Si es una web de contratación de cuidadores, el histórico de cuidados. Si es una web de un gimnasio, los comentarios que pone a cada clase. Esta funcionalidad es importante porque la web debe implementar los mecanismos de seguridad adecuados para que sólo el usuario que ha creado un elemento (su dueño)

pueda borrarlo o editarlo.

- **Imágenes:** La web tiene que permitir la subida de imágenes desde el navegador web. Por ejemplo, como avatar de los usuarios, fotos de productos, etc.
- **Gráficos:** Se deberán usar gráficos (charts) para mostrar algún tipo de información en la web. Por ejemplo:
 - o Web de gestión de torneos o ligas: Gráfica de líneas con la puntuación/clasificación de los equipos a lo largo del tiempo.
 - o Web de gestión de cursos: Número de alumnos registrados en cada uno de los cursos.
 - o Web de cuidado de niños: Gráfica de líneas con las jornadas realizadas por mes para un cuidador.
 - o Web de gestión de exámenes: Gráfica de barras con el número de alumnos por asignatura.
- **Tecnología complementaria:** Se deberá hacer uso de alguna funcionalidad que se implemente con alguna tecnología o librería que no se haya visto en el material de la asignatura. Por ejemplo:
 - o Envío de correos a los usuarios.
 - o Generación de PDFs. (facturas, entradas, etc.)
 - o Uso de websockets para implementar edición o avisos en tiempo real (chat, pizarra compartida, etc.)
 - o Uso de mapas de GoogleMaps / OpenStreetMap para posicionar elementos de la aplicación web (restaurantes, pedidos, etc.)
 - o Uso de una API REST de algún servicio externo (Información meteorológica, libros, películas, etc....).
- **Algoritmo o consulta avanzada:** La aplicación web deberá ofrecer alguna funcionalidad que requiera la implementación de un algoritmo o un tratamiento avanzado sobre los datos que gestiona. No basta con que las entidades se puedan crear, modificar, actualizar y borrar. Por ejemplo:
 - o Si se opta por una web que permita hacer el seguimiento de una liga de fútbol, la clasificación se deberá calcular de forma automática a medida que se vayan registrando los resultados los partidos.
 - o Si se opta por una web de contratación de cuidadores para niños, se implementará un sistema de valoraciones, de forma que la búsqueda tenga en cuenta las valoraciones (mejor valorados primero), o una búsqueda basada en la distancia a la que puede ir el cuidador, etc.
 - o Si es una página web de venta de productos, se deberá implementar un sistema de ofertas personalizadas en base a los productos que haya comprado previamente el

usuario (por ejemplo, mostrar como recomendado un producto de la categoría que más haya comprado el usuario en el pasado).

- o Si se diseña una página de gestión de exámenes, se deberán poder analizar las posibles restricciones que existan (que un mismo alumno no tenga dos exámenes el mismo día, que un profesor no tenga exámenes en el mismo momento, etc).

Documentación

Se creará un fichero README.md usando el formato Markdown en la raíz del repositorio de GitHub. Este fichero se visualizará al entrar en la web de GitHub de ese repositorio y tendrá que estar correctamente formateado usando secciones, subsecciones, tablas, código fuente como texto en formato monoespaciado, imágenes, etc.

La documentación se podrá escribir en castellano, aunque se recomienda que se haga en inglés.

El contenido del fichero README.md deberá contener la siguiente información:

- Nombre de la aplicación web.
- Integrantes del equipo de desarrollo: Nombre, Apellidos, correo oficial de la universidad y cuenta en GitHub.
- Si se utiliza trello o cualquier otra herramienta para la coordinación del equipo, deberá ser pública y se incluirá el link de la misma.
- Una sección describiendo cada uno de los aspectos principales de la aplicación web:
 - o **Entidades:** Es necesario indicar las entidades principales que gestionará la aplicación. No es necesario definir sus atributos, aunque si las relaciones que existan entre ellas.
 - o **Permisos de los usuarios:** Describir someramente los permisos de cada uno de los tipos de usuario. Es importante indicar de qué entidades es dueño el usuario.
 - o **Imágenes:** Indicar qué entidades tendrán asociadas uno o varias imágenes por cada objeto/registro.
 - o **Gráficos:** Qué información se mostrará usando gráficos. De qué tipo serán los gráficos (líneas, barras, tarta, etc).
 - o **Tecnología complementaria:** Qué tecnología complementaria se empleará.
 - o **Algoritmo o consulta avanzada:** Indicar cuál será el algoritmo o la consulta avanzada que se implementará.

Preparación 2: Maquetación de páginas con HTML y CSS

En la preparación 2 se debe definir el esquema de navegación de la web y la maquetación (HTML y CSS) de las páginas más importantes de la web. Se deben incluir las páginas de los administradores (que tienen permisos para editar toda la información de la web).

Para realizar el diseño, se utilizarán datos de ejemplo. Por ejemplo, si la aplicación web es una tienda virtual, para diseñar la página principal, se utilizarán productos de ejemplo con sus imágenes, descripciones, etc. Esta información de ejemplo puede copiarse de cualquier aplicación web de temática similar.

Para el diseño web se utilizará un framework CSS (Bootstrap¹ usando una plantilla gratuita o comercial o Material Design²). El objetivo es que la aplicación web tenga un aspecto profesional.

En los casos en los que sea posible, algunos enlaces de las pantallas permitirán simular la navegación real por la página. Por ejemplo, si la aplicación es una tienda, cuando se pulse el enlace para ver la información del producto en detalle, se debería navegar a una página de detalle de un producto (no tiene por qué ser el producto concreto). No obstante, en esta práctica los formularios no funcionarán.

Esta práctica no es evaluable.

Funcionalidades de la aplicación web

La aplicación deberá tener mínimo las siguientes características:

- Deberán existir páginas para crear, editar y visualizar las entidades, incluido el usuario con su página de perfil.
- Se deberán poder eliminar los elementos de las entidades.
- Deberán existir una página de login y una de registro de usuario.

Documentación

Se añadirá a la información de la preparación 1 en el README del repositorio la siguiente documentación:

- **Capturas de las pantallas:** Se incluirán capturas de pantalla de cada una de las páginas principales que hayan sido maquetadas. Se acompañarán con una breve descripción en

¹ <https://getbootstrap.com/>

² <https://github.com/material-components/material-web>

cada una de ellas (un párrafo como mucho).

- **Diagrama de navegación:** Para mostrar la navegación se creará un diagrama en el que se indicará desde qué página se puede navegar hasta otras páginas. Para ello, las páginas del diagrama pueden ser capturas de pantalla en miniatura de las maquetaciones que se hayan realizado. A continuación, se muestra un ejemplo de diagrama de navegación:



- **Vídeo:** Se deberá crear un vídeo (con OBS o similares), subir ese vídeo a YouTube y enlazar el vídeo en el inicio del README. Este vídeo tendrá una duración máxima de 5 minutos y mostrará las principales funcionalidades de la aplicación web. Uno de los alumnos deberá grabarse la voz describiendo qué está haciendo según vaya interactuando con la aplicación web. El vídeo deberá mostrar, como mínimo, lo que se indique en la rúbrica y todas las páginas de la aplicación. Para ello, recomendamos utilizar un archivo SRT para indicar cada una de las partes de la rúbrica como subtítulos.

Práctica 1: Web con HTML generado en servidor y AJAX

En la práctica 1 se debe implementar la aplicación web completamente funcional.

Funcionalidades de la aplicación web

La aplicación deberá tener las siguientes características:

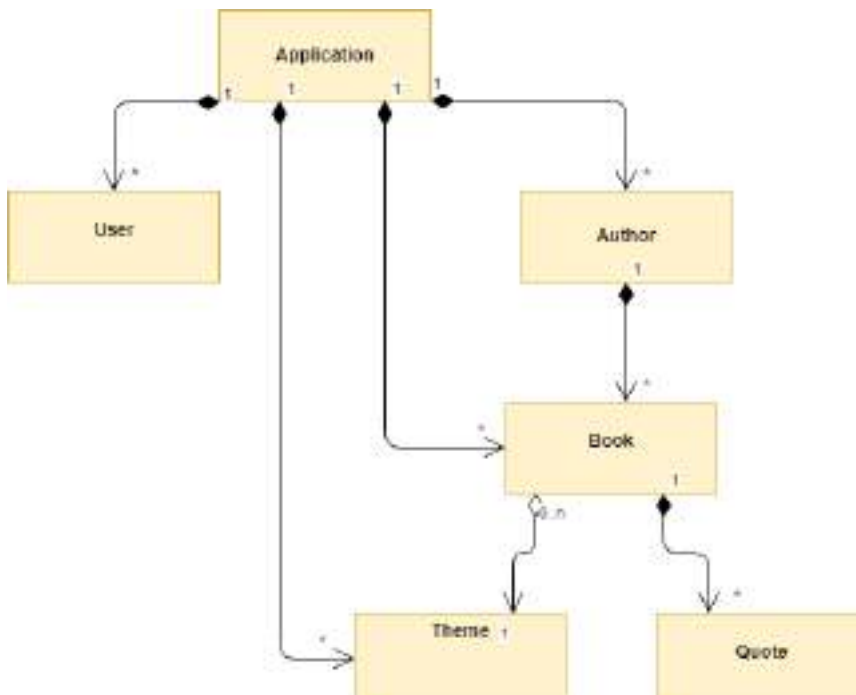
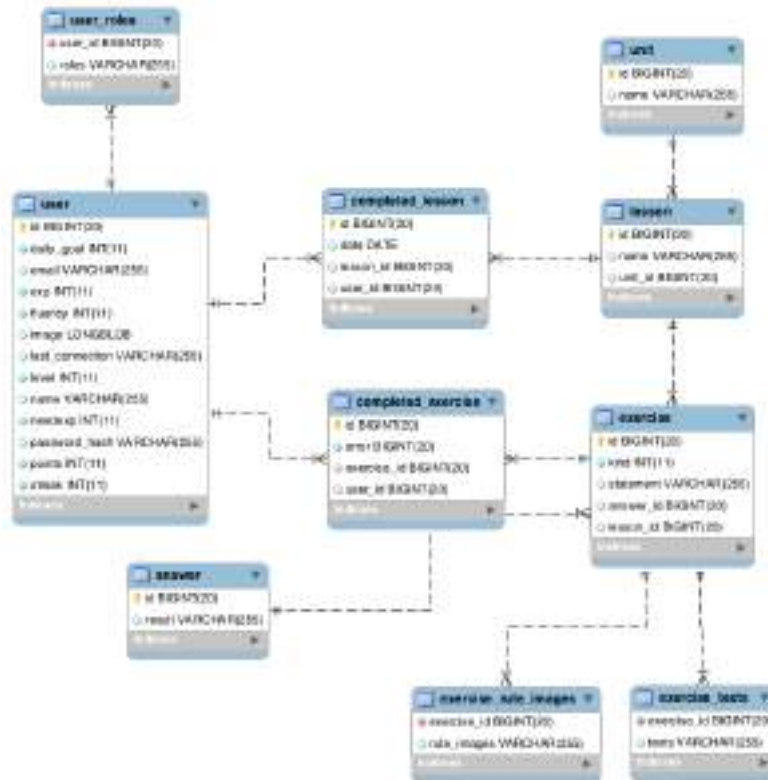
- **Aplicación completa:** La aplicación web se desarrollará con SpringBoot y una base de datos MySQL. Es fundamental que todas las funcionalidades implementadas estén completas. Es preferible reducir el alcance inicial antes que entregar características a medio hacer. Por ejemplo, no tiene sentido tener el acceso a la base de datos correctamente desarrollado si la mitad de las páginas permanecen sin terminar. En resumen, es mejor presentar menos funcionalidades, pero totalmente operativas, que un conjunto más amplio pero incompleto. El profesor evaluará si la reducción es razonable o si será necesario recuperar la práctica.
- **Datos de ejemplo:** Se cargarán datos de ejemplo en la base de datos al ejecutar la aplicación. Estos datos de ejemplo estarán especificados en el código Java. Los datos e imágenes deberán ser representativos del tipo de aplicación web que se esté desarrollando: libros, restaurantes, productos, etc. Esta información se puede obtener de otras webs.
- **Páginas de error:** Cuando se intente acceder a una URL inexistente o se produzca un error en el servidor se deberá generar una página de error que tenga el mismo estilo gráfico que el resto de las páginas de la aplicación.
- **Paginación:** Todas las páginas que potencialmente vayan a mostrar más de 10 elementos deberán mostrar únicamente los 10 primeros y se permitirá al usuario cargar más. Para la carga de más elementos, se mostrará un botón o enlace de “Más resultados”. Al pulsar este botón, un código JavaScript realizará una llamada AJAX al servidor y cargará los siguientes 10 elementos, que se añadirán en la página actual a los elementos ya existentes. Pulsar el botón o enlace de “Más resultados” no debe recargar la página completa en el navegador web. El servidor web devolverá el HTML que será incrustado usando JavaScript en la página web previamente cargada. Durante el tiempo que dure la carga de elementos, en la página deberá aparecer una animación de espera (spinner). Se puede ver un ejemplo de este funcionamiento en la página de Google Images.

- **Usuarios:** La aplicación gestionará los usuarios usando Spring Security. Si un usuario intenta acceder a una URL sobre la que no tiene permisos, se debería mostrar un error de acceso. Por ejemplo, si un usuario intenta editar un registro del que no es dueño, se deberá mostrar un error. El administrador tendrá una contraseña especificada en el código java. Existirán dos usuarios registrados precargados en la base de datos. Se deberá ofrecer un mecanismo para que un usuario pueda registrarse en la aplicación web con un formulario de registro.
- **Código fuente:** Todo el código fuente del proyecto SpringBoot se guardará en una carpeta "backend" del proyecto. Es decir, el fichero pom.xml estará dentro de la carpeta "backend", no en la raíz del repositorio.
- **Seguridad:** La aplicación web deberá servirse por https en el puerto 8443.
- **Imágenes en base de datos:** Para facilitar el despliegue de la aplicación en entornos restringidos, las imágenes se guardarán en la base de datos en vez de en el sistema de ficheros.

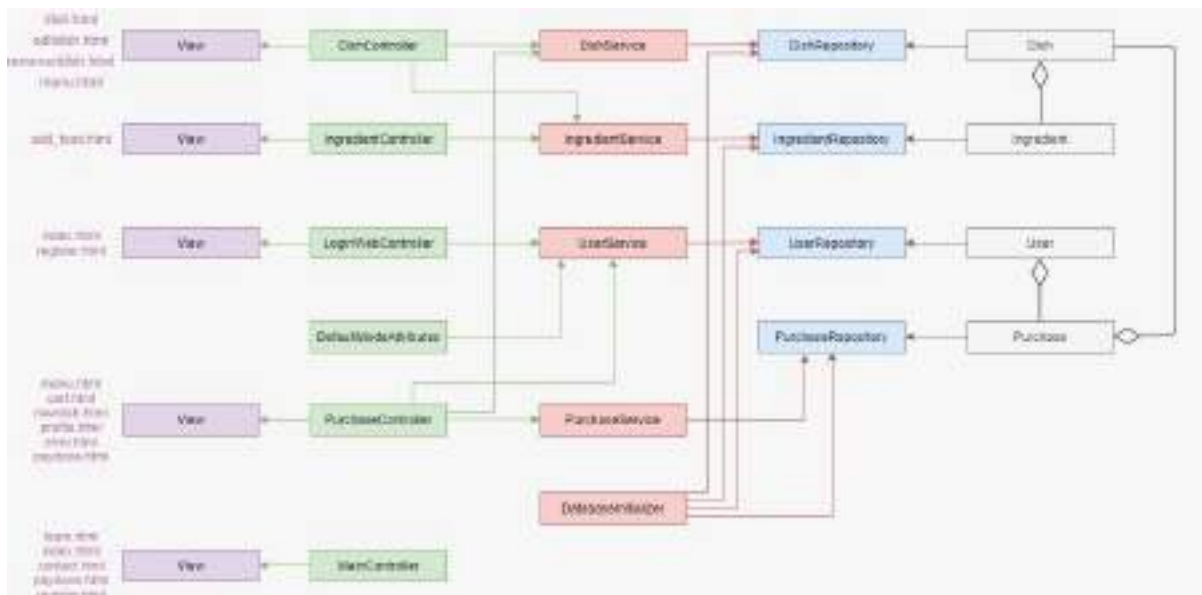
Documentación

En el README se deberá incluir la siguiente información:

- **Navegación:** Se actualizarán las capturas de pantalla de las páginas principales de la aplicación que se hicieron en la práctica 1. En caso de que haya cambiado la navegación, se deberá actualizar también el diagrama de navegación.
- **Instrucciones de ejecución:** Se indicará qué pasos deben seguirse para poder descargar el código del repositorio, construir y ejecutar la aplicación. También se debe especificar cuáles son los requisitos (versión de Java, versión de MySQL, maven, etc.). Las instrucciones se especificarán preferentemente como comandos a ejecutar por la línea de comandos. En caso de que no sea posible, se indicará cómo instalar/configurar aplicaciones de forma interactiva.
- **Diagrama con las entidades de la base de datos:** Se incluirá un diagrama con las entidades de la base de datos, sus campos y las relaciones entre ellas. Se podrá incluir un diagrama entidad-relación de la base de datos o un diagrama UML de las clases Java. A continuación se muestran algunos ejemplos de diagramas:



- **Diagrama de clases y templates:** Se creará un diagrama de clases de la aplicación. No se incluirán ni atributos ni métodos en las clases. Se mostrarán las relaciones entre las clases (asociación, composición y herencia) y se diferenciará claramente qué clases son @Controller, @Service, Repository, clases del dominio (entidades) u otro tipo de clases. Para ello se puede usar un código de colores, una distribución de las clases por partes u otro mecanismo. En este diagrama también se incluirán los ficheros que contienen los templates y se indicará con qué @Controller se relacionan. A continuación, se muestra un diagrama de ejemplo que usa los colores para indicar el tipo de elemento.



- **Participación de miembros:** Se debe incluir una sección en el README en la que cada miembro del equipo indique su participación en esa práctica con:
 - o Descripción textual de las tareas realizadas en la práctica.
 - o Listado de los 5 commits más significativos durante la práctica (enlazados a dichos commits en GitHub).
 - o Listado de los 5 ficheros en los que más haya participado (enlazados a dichos ficheros en GitHub).
- **Vídeo:** Se deberá grabar la pantalla mostrando las funcionalidades de la aplicación web (utilizando OBS o similares). Uno de los alumnos deberá grabarse la voz describiendo qué está haciendo según vaya interactuando con la aplicación web. El vídeo deberá grabarse siguiendo las instrucciones de la rúbrica. Se deberá subir ese vídeo a YouTube y enlazar el vídeo en el inicio de la sección de esta práctica en el README.

Evaluación

El día de la evaluación cada alumno deberá abrir su práctica en el editor de código del Escritorio de Desarrollo de MyApps. Para evitar problemas y no perder tiempo en realizar esa tarea el día de la evaluación, el alumno puede descargar el código de su práctica en la unidad R y verificar que lo puede cargar en el editor de desarrollo.

Con el código abierto en el editor, el alumno tendrá que realizar una serie de tareas que serán detalladas al iniciar la clase. Las tareas pueden consistir en responder algunas preguntas sobre el funcionamiento del código o que realice alguna modificación de la funcionalidad existente o añadir funcionalidad nueva.

Práctica 2: API REST, despliegue con Docker y despliegue remoto

En la práctica 2 se deberá implementar una API REST para la aplicación, se deberá empaquetar y distribuir usando la tecnología Docker y finalmente hacer un despliegue en una maquina remota.

API REST

La implementación de la API REST se tendrá que realizar teniendo en cuenta los siguientes aspectos:

- La aplicación ofrecerá a la misma vez la interfaz web y la API REST. Esto permite que la web siga disponible y que clientes móviles u otros servicios puedan acceder a la funcionalidad proporcionada por la web. En <https://github.com/codigus-formacion/daw-public/tree/master/ejemplo-practica2> se puede encontrar una aplicación de ejemplo con interfaz web y API REST que puede servir de ejemplo.
- Todas las URLs de la API REST comenzarán con "/api/v1/" para diferenciarse de las rutas con las que se accede a páginas de la web.
- La API REST debe diseñarse teniendo en cuenta las buenas prácticas:
 - Los métodos http GET, PUT, POST, DELETE deben usarse de forma adecuada para las operaciones de consulta, modificación, creación y borrado.
 - Las URLs deben identificar recursos. Cada tipo de recurso se identificará en inglés y en plural.
 - Los códigos de estado de respuesta deben usarse de forma adecuada.
 - Las operaciones de creación deberán devolver la header "Location" cuyo valor es la URL con la que se puede obtener la representación del recurso recién creado.
 - Las operaciones de la API REST que permitan filtrar o buscar elementos deberán tener los criterios de filtrado o búsqueda como parámetros de la URL (después del "?").
 - No deberán aparecer verbos en la URLs de la API REST. Es decir, no están permitidas URLs del tipo `https://server.com/api/v1/createPost`.
- Toda funcionalidad que se pueda realizar desde la interfaz web debe poder realizarse desde la API REST. Es decir, si se implementara una aplicación SPA o una aplicación móvil basada en la API REST debería poder realizar cualquier operación disponible en la web.
- La API REST tendrá que implementar los mismos roles de usuario que la aplicación web. Es decir, un usuario no autenticado no podrá realizar operaciones que sólo están disponibles para un usuario autenticado. Lo mismo ocurre para los administradores.
- Para facilitar la implementación, se desactivará la protección contra CSRF en la API REST (en la web seguirá estando disponible).

- Los listados que estén paginados en la aplicación web también lo estarán en la API REST. Los parámetros usados para configurar la paginación serán los mismos que los usados en la aplicación web ("?page=1").
- Se deberá incluir en la raíz del repositorio de código una colección de peticiones Postman (`api.postman_collection.json`) con ejemplos de peticiones a todos los *endpoints* de la API REST que deberá ser completamente funcional con los datos de ejemplo de la aplicación. Es decir, deberán incluir la URL, los parámetros y el cuerpo de la petición. De esta forma, será muy sencillo verificar el comportamiento de la API REST desde Postman. Las peticiones de esta colección deberán poder ejecutarse cuando la API REST esté disponible en desarrollo (localhost) o cuando se despliega en un servidor remoto (una IP específica). Para ello se usará la variable `{{baseUrl}}` del entorno (*environment*).
- La API REST deberá estar documentada con OpenAPI generado mediante SpringDoc.
- Para evitar la duplicación de código y para que la lógica de negocio no esté acoplada a los controladores, se deberá implementar un `@Service` que será usado tanto por el `@Controller` como por el `@RestController`.
 - Ese `@Service` será el que acceda a los repositorios.
 - Un controlador nunca debería acceder a un repositorio directamente.
 - Un controlador REST no debe usar las entidades para procesar los datos de entrada o salida. Para ello debe usar DTOs.
 - Se puede usar como base el ejemplo disponible en <https://github.com/codigus-formacion/daw-public/tree/master/ejemplo-practica2>.

Pasos recomendados para añadir la API REST a la Práctica 2 en base a la Práctica 1:

- En caso de que exista, mover la lógica de negocio de los controladores a los servicios (para que pueda ser reutilizada entre los controladores web y REST).
- Crear los DTOs para que las entidades puedan gestionarse en los controladores REST.
- Implementar los controladores REST y la colección de Postman.
- Añadir la seguridad a los controladores REST manteniendo la misma seguridad de la aplicación web (aunque se requieran cambios en el código) y actualizar la colección Postman para autenticar las peticiones.
- Añadir la documentación de la API REST.

Empaquetado con Docker

El empaquetado con Docker deberá implementarse con las siguientes características:

Imagen Docker de la aplicación:

- La aplicación Spring deberá empaquetarse en una imagen Docker.
- Al crear el contenedor Docker partiendo de la imagen todas las funcionalidades de la aplicación deberán estar disponibles.
- Al igual que en la ejecución local, la aplicación deberá estar disponible en el puerto 8443 por https.

- Al empaquetar la aplicación con Docker no se puede asumir que existe ninguna herramienta instalada en la máquina como Maven, Java Development Kit, etc.
- Cada alumno deberá publicar la imagen de la aplicación en DockerHub en una cuenta propia. Esa cuenta se usará en el examen de la práctica. Si ya disponen de una cuenta personal la pueden usar, o pueden crear una nueva para la asignatura.

Docker compose:

- Se usará docker compose para ejecutar la aplicación completa formada por el contenedor de la base de datos y el contenedor de la aplicación.
- Para la base de datos se usará la imagen oficial de MySQL de DockerHub.
- Para que la aplicación web se inicie correctamente una vez que la base de datos ha arrancado y es accesible se hará uso de la propiedad de `healthcheck`.
- El fichero `docker-compose.yml` configurará el esquema de la base de datos, la ruta de la base de datos y la contraseña de root de la base de datos tanto en el servicio de la aplicación como en el de la base de datos. Para ello, usará la capacidad de especificar variables de entorno. Conviene revisar la documentación sobre las variables de entorno de la imagen Docker de MySQL y las variables de entorno `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME` y `SPRING_DATASOURCE_PASSWORD` que permiten configurar la base de datos en una aplicación implementada con SpringBoot.
- Cada alumno debe publicar el `docker-compose.yml` en su cuenta de DockerHub como un OCI Artifact de forma que la aplicación se pueda ejecutar con un único comando (que descarga el `docker-compose.yml` y luego las imágenes Docker).

Código fuente:

En el repositorio de GitHub se deberá crear una carpeta `/docker` que tendrá los siguientes ficheros:

- **Dockerfile:** Fichero utilizado para crear la imagen docker de la aplicación.
- **create_image.sh/.bat/ps1:** Script de bash, bat o power shell que construirá la imagen docker de la aplicación desde el código fuente. El script se ejecutará una vez clonado el repositorio de código. El único requisito para que el script funcione correctamente debería ser que el sistema tenga instalado Docker. No se deberá usar el JDK del sistema para compilar la aplicación Java. Este script no debe publicar la imagen en DockerHub. Se debe pasar el nombre de la imagen como parámetro.
- **publish_image.sh/.bat/.ps1:** Publicará la imagen en la cuenta de DockerHub especificada como parámetro.
- **docker_compose.yml:** Fichero que al ejecutarse con el comando “docker compose up” se descargue la imagen de la aplicación y de la base de datos de DockerHub y ejecute la aplicación en el puerto 8443 local de forma que sea accesible en <https://localhost:8443/>.
- **publish_docker-compose.sh/.bat/.ps1:** Publicará el `docker-compose.yml` en la cuenta de DockerHub especificada como parámetro.

Despliegue remoto

La aplicación web deberá desplegarse en un servidor remoto. Se proporcionará acceso mediante SSH a un servidor linux a cada grupo. Este servidor tendrá abierto el puerto 8443 para que se pueda acceder a la aplicación.

Los alumnos tendrán que instalar Docker y Docker Compose y posteriormente desplegar la aplicación usando el docker-compose publicado en de DockerHub. Se puede usar la cuenta de cualquiera de los alumnos del grupo.

La IP de la máquina y las credenciales de acceso se proporcionarán a los alumnos mediante correo electrónico.

Aviso: Sólo se tiene acceso a la máquina (tanto por SSH como vía web) desde la red wifi de la universidad o desde una máquina en myApps.

Se deberán realizar los cambios necesarios en la aplicación para que se pueda desplegar en dos modos diferentes:

- Modo inicialización: Para crear las tablas y cargar los datos de ejemplo.
- Modo ejecución: Que no modifique el esquema de la base de datos y no cargue los datos de ejemplo.

Para poder configurar estos dos modos se deben seguir los siguientes pasos:

- Se debe modificar el servicio **DataBaseInitializer** para que cargue los datos únicamente cuando la base de datos se encuentre vacía. Para ello, se debe realizar una consulta `.count()` utilizando, por ejemplo, el repositorio de usuarios, y **comprobar si el número de registros es igual a cero**. Si no existe ningún usuario en la base de datos, se procederá a **insertar todos los datos de ejemplo**, en caso contrario, **no se deberán cargar dichos datos**.
- Se debe configurar la variable de Spring Boot `SPRING_JPA_HIBERNATE_DDL_AUTO` en el fichero `docker-compose.yml`. Esta variable permite definir el modo de funcionamiento de la gestión del esquema de la base de datos. Por defecto, se establecerá el valor **"none"**, de modo que la aplicación no cree ni modifique automáticamente el esquema de la base de datos. Sin embargo, se debe parametrizar mediante una variable de entorno para que su valor pueda modificarse (revisar el tema de Docker Compose, concretamente la sección en la que se explica cómo inyectar variables de entorno en los OCI Artifacts).
- Por último, al desplegar la aplicación utilizando el docker-compose publicado en Docker Hub, se debe indicar, **únicamente en el primer arranque de la aplicación**, que utilice el modo **"create"**. De este modo, se generará el esquema de la base de datos y se cargarán los datos de ejemplo. Cuando se detenga la aplicación y se vuelva a iniciar, se deberá restablecer el valor por defecto **"none"**, de forma que el esquema de la base de datos no se vuelva a crear.

Documentación

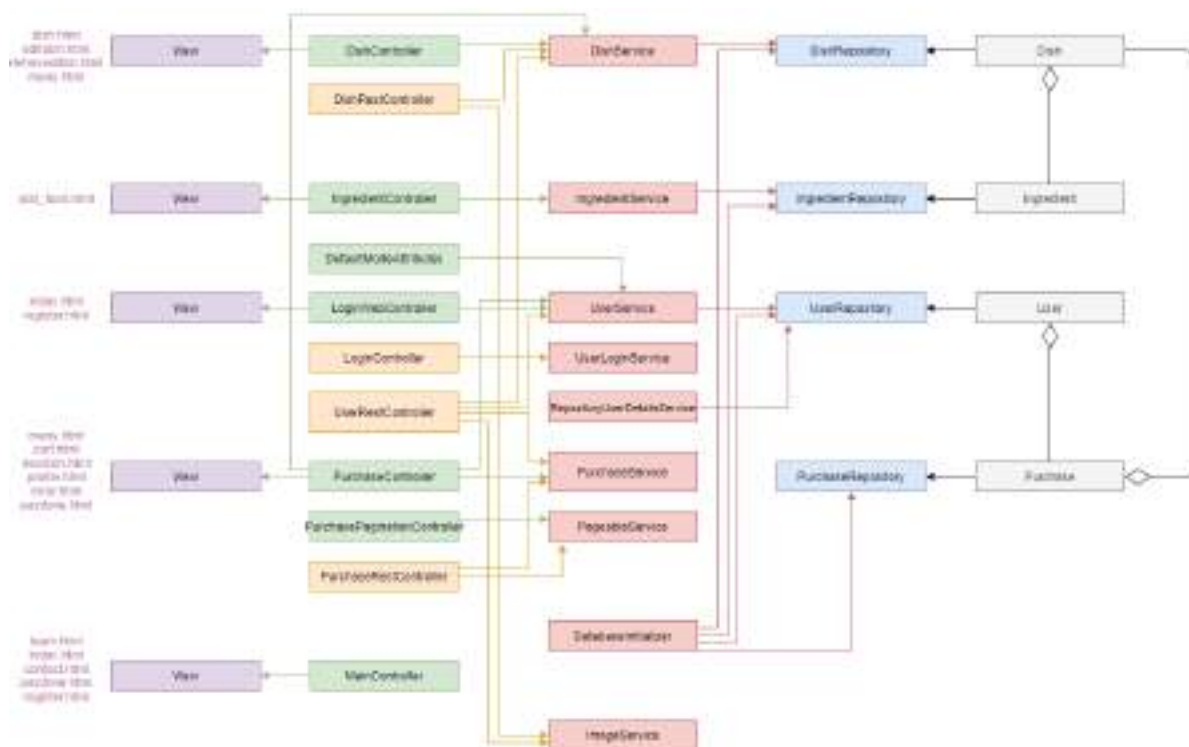
En el README se deberá incluir la siguiente información:

- **Documentación de la API REST:** La documentación de la API REST se alojará en la carpeta /api-docs de la raíz del repositorio y estará formada por la especificación OpenAPI (en un fichero api-docs.yaml) y por un fichero HTML generado a partir de esa documentación (api-docs.html).

Se debe definir la documentación de forma adecuada en el código Java con SpringDoc de forma que las descripciones del documento HTML sean adecuadas.

En el README se debe poner un enlace a la especificación OpenAPI (fichero /api-docs/api-docs.yaml). También se debe poner un enlace a la documentación en HTML, pero para que se visualice correctamente en el browser se deberá generar el enlace con el servicio <https://raw.githubusercontent.com> para que el fichero HTML se visualice correctamente con estilos (no como un fichero del repositorio).

- **Actualización de diagrama de clases:** Se actualizará el diagrama de clases y plantillas incluyendo las nuevas clases @RestController. Se prestará especial atención a que los @Service compartidos entre los @Controller y los @RestController aparezcan correctamente reflejados en el diagrama. No se mostrarán en el diagrama los DTOs. A continuación, se muestra un ejemplo de este tipo de diagrama:



- **Instrucciones de ejecución de la aplicación empaquetada con Docker:** Instrucciones de ejecución usando el docker-compose desplegado en DockerHub. Se deberán indicar los requisitos para que se pueda ejecutar, el comando necesario y cómo acceder a la aplicación cuando esté lista para ser usada. Se puede utilizar la cuenta de cualquiera de los alumnos.

- **Documentación para construcción y publicación:** Se indicarán los requisitos necesarios para construir la imagen docker de la aplicación y cómo ejecutar los scripts necesarios para construir y publicar la imagen Docker en DockerHub. También se documentará cómo ejecutar el script de publicación del docker-compose.yml.
- **Documentación para desplegar en servidor remoto:** Se deberán describir los requisitos y comandos para poder desplegar la aplicación en el servidor remoto.
- **La URL de la aplicación desplegada en el servidor remoto:** Se deberá incluir la URL de la máquina virtual donde se ha desplegado la aplicación. También se deberán incluir las credenciales de los usuarios de ejemplo (incluyendo el administrador).
- **Participación de miembros:** Se debe incluir una sección en el README en la que cada miembro del equipo indique su participación en esa práctica con:
 - o Descripción textual de las tareas realizadas en la práctica.
 - o Listado de los 5 commits más significativos durante la práctica (enlazados a dichos commits en GitHub).
 - o Listado de los 5 ficheros en los que más haya participado (enlazados a dichos ficheros en GitHub).
- **Vídeo:** Se deberá grabar la pantalla mostrando las funcionalidades de la aplicación web (utilizando OBS o similares). Uno de los alumnos deberá grabarse la voz describiendo qué está haciendo según vaya interactuando con la aplicación web. El vídeo deberá grabarse siguiendo las instrucciones de la rúbrica. Se deberá subir ese vídeo a YouTube y enlazar el vídeo en el inicio de la sección de esta práctica en el README.

Evaluación

El día de la evaluación cada alumno deberá cargar su práctica en MyApps. Para evitar problemas y no perder tiempo en realizar esa tarea el día de la evaluación, el alumno puede descargar el código de su práctica en la unidad R antes de la entrega y verificar que puede realizar cambios en la aplicación, empaquetarla y acceder al servidor remoto.

Con el código cargada en MyApps, el alumno tendrá que realizar una serie de tareas que serán detalladas al iniciar la clase. Las tareas pueden consistir en responder algunas preguntas sobre el funcionamiento del código, realizar alguna modificación de la funcionalidad existente o añadir funcionalidad nueva.

Práctica 3: Web con arquitectura SPA

En la práctica 3 se debe implementar la aplicación web como cliente SPA con React.

Funcionalidades de la aplicación SPA

Esta práctica se implementará teniendo en cuenta los siguientes aspectos:

- Este cliente debe tener exactamente la misma funcionalidad que la web implementada en la práctica 1. Aunque no es necesario que la interfaz gráfica sea exactamente igual mientras se puedan realizar las mismas operaciones. No obstante, se tendrá que seguir manteniendo el mismo estilo general.
- La aplicación web se podrá usar desde el interfaz web original (implementado en la práctica 1) o desde el cliente SPA de esta práctica. Si se editan datos desde la interfaz original, deberán ser visibles desde la aplicación React y viceversa.
- La web con arquitectura tradicional (MVC) será accesible desde la URL principal (<https://localhost:8443/>) y la web SPA desde la URL (<https://localhost:8443/new/>).
- Las pantallas se implementarán usando las librerías gráficas de react-bootstrap respetando el estilo gráfico de la web de la práctica 1.
- Todo el código JavaScript implementado en la práctica 1 tendrá que volver a implementarse con TypeScript dentro del proyecto de React. En concreto, se deberá implementar el mismo mecanismo de paginación de la práctica 1 basado en AJAX, pero usando la API REST y mostrando las páginas con React.
- La información del usuario autenticado debe gestionarse mediante un store de Zustand. Además, las peticiones a la API REST deben manejarse utilizando un clientLoader, acompañado de un spinner global para mejorar la experiencia de usuario durante las cargas.
- Publicación:
 - o La aplicación React se publicará como otro recurso estático más de la aplicación web SpringBoot (en la carpeta new). De esa forma, se debería poder acceder a la aplicación cuando se acceda a la ruta <https://localhost:8443/new/>.
 - o Como la construcción de la aplicación deberá basarse en Docker (sin usar herramientas nativas en la máquina), deberá ampliarse el Dockerfile de la práctica 2

para que construya también la aplicación React. Para ello, debe utilizarse la imagen oficial de Node.js como otro paso más del Docker Multi-stage en la que se ejecutarán los comandos para construir.

- o Se deberá desplegar la aplicación actualizada en el servidor de la universidad. La web MVC tradicional será accesible en https://IP_MAQUINA:8443/ y a la web SPA en https://IP_MAQUINA:8443/new/.

- **Código fuente:** Todo el código fuente del proyecto React se guardará en una carpeta “frontend” del repositorio.

Documentación

En el README se deberá incluir la siguiente información:

- **Preparación del entorno de desarrollo:** Se añadirá a las instrucciones cómo instalar y configurar el entorno de desarrollo para poder desarrollar la aplicación SPA con React.
- **Diagrama de elementos de la SPA:** Reflejará los componentes, layout, stores, servicios y/o contextos usados en la aplicación React y sus relaciones. El diagrama deberá diferenciar claramente cada tipo de elemento.
- **Participación de miembros:** Se debe incluir una sección en el README en la que cada miembro del equipo indique su participación en esa práctica con:
 - o Descripción textual de las tareas realizadas en la práctica.
 - o Listado de los 5 commits más significativos durante la práctica (enlazados a dichos commits en GitHub).
 - o Listado de los 5 ficheros en los que más haya participado (enlazados a dichos ficheros en GitHub).
- **Vídeo:** Se deberá grabar la pantalla mostrando las funcionalidades de la aplicación web (utilizando OBS o similares). Uno de los alumnos deberá grabarse la voz describiendo qué está haciendo según vaya interactuando con la aplicación web. El vídeo deberá grabarse siguiendo las instrucciones de la rúbrica. Se deberá subir ese vídeo a YouTube y enlazar el vídeo en el inicio de la sección de esta práctica en el README.

Evaluación

El día de la evaluación cada alumno deberá cargar su práctica en MyApps. Para evitar problemas y no perder tiempo en realizar esa tarea el día de la evaluación, el alumno puede descargar el código de su práctica en la unidad R antes de la entrega y verificar que puede realizar cambios en la aplicación, empaquetarla y acceder al servidor remoto.

Con el código cargada en MyApps, el alumno tendrá que realizar una serie de tareas que serán detalladas al iniciar la clase. Las tareas pueden consistir en responder algunas preguntas sobre el funcionamiento del código, realizar alguna modificación de la funcionalidad existente o añadir funcionalidad nueva.